

2. TASK

2.1 Sentiment Analysis

2.1.1 모델 구조 및 튜닝 방식

감정 분류기에서는 앞서 1.3절에서 설명한 Task-specific Head 구조를 그대로 사용하였다. 입력 문장은 GPT-2를 통해 인코딩되며, 마지막 토큰의 hidden state를 문장 벡터로 활용한 뒤, 2층 MLP 분류기(Linear → ReLU → Linear)를 거쳐 5개의 감정 클래스(0~4)에 대한 logits을 출력한다.

본 감정 분류 모델에서는 사전 학습된 GPT-2를 고정(freeze)한 상태로 사용하고, 그 위에 추가된 분류기(classifier)만 학습하는 방식을 적용하였다. 구체적으로는 모델 초기화 시 GPT-2의 모든 파라미터에 대해 `requires_grad = False`로 설정하고, 분류기 계층만 gradient가 계산되도록 구성하였다. 따라서 학습 과정에서는 사전 학습된 언어모델의 표현은 유지된 채, 분류기 파라미터만 업데이트되어 task-specific 정보를 학습하게 된다.

```
for param in self.gpt.parameters():  
    param.requires_grad = False
```

```
# 분류기만 학습 대상 (2-layer MLP)  
self.classifier_1 = nn.Linear(self.gpt.config.hidden_size, hidden_dim)  
self.relu = nn.ReLU()  
self.classifier_2 = nn.Linear(hidden_dim, num_labels)
```

2.1.2 결과

```
🔄 Training Sentiment Classifier on SST...  
Evaluating on SST...  
load model from /content/drive/MyDrive/unzipped_folder/sst-classifier.pt  
load 1101 data from /content/drive/MyDrive/unzipped_folder/data/ids-sst-dev.csv  
eval: 100%|██████████| 138/138 [00:02<00:00, 50.33it/s]  
DONE DEV  
eval: 100%|██████████| 277/277 [00:05<00:00, 48.87it/s]  
DONE Test  
dev acc :: 0.472
```

학습된 모델을 validation set에 평가한 결과, 정확도(Accuracy)는 47.2%로 나타났다.

이 결과는 사전 학습된 GPT-2 표현은 유지하되 분류기만 업데이트한 설정의 성능을 나타내며, 모델이 텍스트 표현보다는 추가된 MLP 분류기에 의존하여 감정을 분류했음을 의미한다. 성능 향상을 위해서는 task-specific prompt를 입력에 삽입하는 prompt-tuning 등의 fine tuning 기법이 추가로 도입될 수 있다.

2.2 paraphrase detection

2.2.1 서론 및 튜닝 방법

본 과제에서는 GPT-2 기반의 Paraphrase Detection 모델에 다양한 구조적 변화를 주어 성능 향상을 시도하였다. 그 중에서도 MLP Head + Prompt Tuning 조합이 가장 높은 Dev Accuracy를 기록하였다.

사용한 튜닝 방식은 MLP head + Prompt Tuning + Full Fine-tuning이다. MLP Head는 1.3절에서 설명한 것처럼 GPT-2의 마지막 hidden 상태 위에 이중 Linear Layer를 쌓은 구조로, 문장 쌍의 유사 여부를 분류하는 역할을 수행한다.

Prompt Tuning은 입력 문장 앞에 명시적인 태스크 지시문을 추가하여, 모델이 문장의 관계를 보다 명확히 이해하도록 유도하는 기법이다. 본 실험에서는 다음과 같은 프롬프트 템플릿을 사용하였다.

```
<task=Paraphrase_Detection> <format=cloze> <output=yes/no>
Question 1: "{sentence1}"
Question 2: "{sentence2}"
Are these asking the same thing?
```

이러한 프롬프트는 기존의 단순 입력 포맷과 달리, 자연어 형태의 구조적 지시어를 통해 모델의 응답 방향성을 유도하며, 사전 학습된 GPT-2가 보다 효과적으로 paraphrase 여부를 판단할 수 있도록 돕는다.

한편, 기존의 Prompt Tuning은 보통 모델 파라미터를 고정(freeze)한 채로 입력 토큰만 조정하는 soft prompt 방식과 결합되는 경우가 많지만, 본 과제에서는 GPT-2 전체를 Fine-tuning 대상으로 포함함으로써, 입력 구조와 내부 표현 학습을 동시에 강화하였다.

2.1.2 결과

정량적 성능 비교

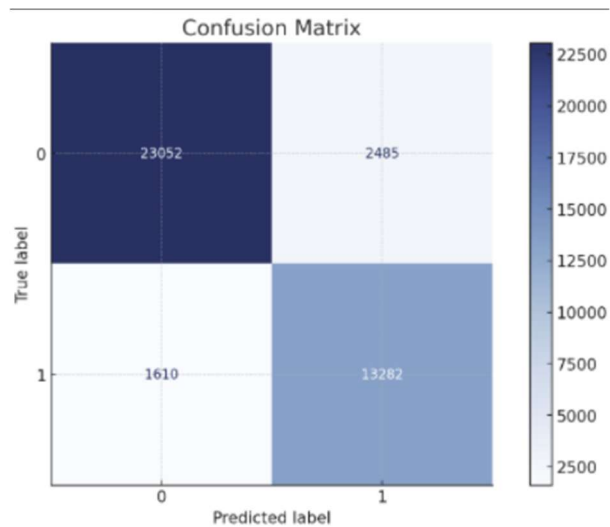
실험 결과 다음과 같은 정량적 성능을 기록하였다.

- Dev Accuracy: 0.899
- Weighted F1 Score: 0.899

이는 Baseline 모델(GPT-2 + Linear Head, Prompt 미사용)의 Dev Accuracy인 0.897보다 소폭 향상된 결과이며, 이미 높은 성능을 보이는 상황에서의 개선이라는 점에서 의미가 있다.

혼동 행렬

예측 결과를 바탕으로 한 혼동 행렬은 다음과 같다



실제값 / 예측값 Paraphrase(1) Not Paraphrase(0)

Paraphrase(1) TP: 13,282 FN: 1,610

Not Paraphrase(0) FP: 2,485 TN: 23,052

paraphrase 문장을 잘못 not paraphrase로 판단한 경우(FN)는 1,610건, 문장을 paraphrase로 오판한 경우(FP)는 2,485건으로, 전체적으로 FP가 더 많은 경향을 보인다. 이는 모델이 일부 not paraphrase 문장을 과도하게 유사하다고 판단하는 경향이 있음을 의미하며, Precision 대비 Recall을 더 개선할 여지가 있는 구조임을 시사한다.

오답 사례 분석

다음은 모델이 잘못 예측한 실제 문장 예시들이다.

예측: Paraphrase (1), 실제: Not Paraphrase (0)

- 모델이 유사하지 않은 문장을 잘못 유사하다고 판단한 사례 (False Positive)

문장1: *How long can you leave cooked chicken wings out?*

문장2: *Can you leave raw chicken out all night?*

→ 단어는 유사하지만 "익힌 닭고기"와 "생 닭"이라는 명확한 차이가 있으며, 보건 기준도 다름.

문장1: *I am a girl. How can I join the Indian Army after BDS?*

문장2: *How do I join the Indian Army as a dentist after BDS?*

→ 문장 구조는 유사하나, "여성 지원 가능 여부"라는 조건이 무시됨

◆ 예측: Not Paraphrase (0), 실제: Paraphrase (1)

모델이 paraphrase 문장을 잘못 not paraphrase 판단한 사례 (False Negative)

문장1: *Are floppy disks still used today?*

문장2: *Are floppy disks still useful?*

→ 표현 방식이 다를 뿐 의미는 거의 동일함.

문장1: *What is Shaoxing vinegar?*

문장2: *What is Shaoxing vinegar used for?*

→ 하나는 정의, 하나는 용도지만 상호 보완적 질문이며 의미가 겹침.

이러한 사례를 통해 확인할 수 있는 오답 경향은 다음과 같다.

- 단어는 겹치지만 의미적 맥락이 다른 경우: False Positive

- 표현은 다르지만 의미는 유사한 경우: False Negative

이는 모델이 표면적 유사성에 치중하거나, 문맥 이해력이 부족함을 시사한다.

성능 향상 위한 향후 보완 전략

Error-aware Data Augmentation

- 오답으로 판별된 샘플을 중심으로 대칭 문장 또는 교란 문장을 생성하여 학습 데이터에 포함.
- 특히 표현만 바꾼 paraphrase 또는 미묘한 의미 차이를 가지는 negative 예시를 강화하여

모델의 예외 처리 능력 향상 기대.

Multi-task Learning 구조 도입

- Paraphrase Detection 외에도 문장 유형 분류(Sentence Intent Classification) 또는 문장 간 관계 추론(NLI) 과 같은 태스크를 병행 학습.
- 다양한 의미 판단 능력을 동시에 학습함으로써, 의미 분별 능력이 내재된 언어 표현을 보다 강인하게 획득 가능.

2.1.3 실패한 실험

학습 전략 및 단계적 실험 구성

본 실험에서는 모델 안정성과 자원 효율성, 그리고 성능 향상을 모두 고려하여, 단계적인 학습 전략을 설계하였다. 초기 5 epoch 동안은 GPT-2 본체는 고정(freeze) 한 채, MLP Head만 학습하였다. Head를 충분히 학습한 후 후반부 5 epoch는 GPT-2 전체를 fine-tune하도록 하였다.

이 방식은 GPU 메모리 사용량을 줄이고, 사전 학습된 GPT-2 표현력을 그대로 활용하면서 빠르게 수렴 가능할 것이라는 기대 아래 설계된 전략이었다.

그 결과

```
save the model to 10-le-05-mlp_prompt-paraphrase.pt
Epoch 4: train loss :: 0.608, dev acc :: 0.711

[!] Unfreezing GPT2 for full fine-tuning

train-5: 100% 35376/35376 [30:39:00-00, 19.30it/s]
eval: 100% 5054/5054 [01:14:00-00, 67.87it/s]
save the model to 10-le-05-mlp_prompt-paraphrase.pt
Epoch 5: train loss :: 0.356, dev acc :: 0.870
train-6: 100% 35376/35376 [30:48:00-00, 19.13it/s]
eval: 100% 5054/5054 [01:14:00-00, 67.80it/s]
save the model to 10-le-05-mlp_prompt-paraphrase.pt
Epoch 6: train loss :: 0.289, dev acc :: 0.883
train-7: 100% 35376/35376 [30:44:00-00, 19.16it/s]
eval: 100% 5054/5054 [01:14:00-00, 67.62it/s]
save the model to 10-le-05-mlp_prompt-paraphrase.pt
Epoch 7: train loss :: 0.221, dev acc :: 0.891
train-8: 100% 35376/35376 [30:44:00-00, 19.18it/s]
eval: 100% 5054/5054 [01:14:00-00, 67.84it/s]
save the model to 10-le-05-mlp_prompt-paraphrase.pt
Epoch 8: train loss :: 0.180, dev acc :: 0.892
train-9: 100% 35376/35376 [30:44:00-00, 19.18it/s]
eval: 100% 5054/5054 [01:14:00-00, 67.75it/s]
Epoch 9: train loss :: 0.146, dev acc :: 0.891
/content/drive/MyDrive/ColabProjects/GPT2/paraphrase_detection.py:154: FutureWarning:
  saved = torch.load(saved_filepath)
Loaded model to test from 10-le-05-mlp_prompt-paraphrase.pt
Loaded 40829 train examples from data/qaora-dev.csv
Loaded 89859 test examples from data/qaora-test-student.csv
eval: 100% 5054/5054 [01:15:00-00, 65.46it/s]
dev paraphrase acc :: 0.892
eval: 100% 5054/5054 [01:15:00-00, 65.46it/s]
```

초기 4개의 epoch 동안 GPT-2를 고정하고 MLP head만 학습했을 때, Dev Accuracy는 최대 0.711에 그쳤다.

이후 5번째 epoch부터 GPT-2 전체를 unfreeze 하여 Full Fine-tuning을 적용하자 성능이 급격히 향상되었으며, Epoch 9에서 Dev Accuracy는 0.891, 최종 Dev Accuracy는 0.892를 기록하였다.

이는 GPT-2의 사전 학습된 표현을 그대로 유지한 채, 출력 부분만 조정하는 방식으로는 문장 간의 미묘한 의미 차이나 문맥적 관계를 충분히 학습하기 어려웠기 때문으로 보인다. 특히 Paraphrase Detection과 같은 정밀한 의미 판단 과제에서는, 전체 모델의 표현 공간을 조정해야만 확실한 분류가 가능하다는 한계를 드러낸 결과였다. 이러한 결과는 Full Fine-tuning이 모델의 표현력을 보다 효과적으로 조정하여 정확도 향상에 직접적인 기여를 했음을 보여준다.

2.3 Sonnet Generation

2.3.1 모델 구조 및 튜닝 방식

본 프로젝트에서는 GPT-2를 백본으로 한 SonnetGPT 모델을 사용하였다. 다음과 같은 세 가지 방식으로 튜닝을 수행하였다:

사용한 튜닝 방식: LoRA, Instruction tuning, Distillation

2.3.2 실험_1 - Distillation 미적용 실험

실험 과정

- 전체 모델을 한 번에 튜닝하면 과적합 및 불안정성이 크므로, LoRA와 Transformer 레이어를 점진적으로 확장해 가며 학습하는 방식이 더 안정적일 것이라 가정한다.
- Loss가 일정 수준에서 정체되는 원인은 모델의 표현력 부족(complexity 한계)으로 추정한다.
- Train Loss와 Validation Loss가 Coupling 돼야 하며, Decoupling 시 과적합으로 간주한다.
- 이전 가중치를 저장하고 로드하여 이어서 학습하며, 실험 간 일관성을 유지한다.

하이퍼 파라미터 설정

- Learning Rate: 기본값 $1e-5$ → 실험적으로 $5e-6$, $1e-3$ 등 변경하며 안정성 탐색
- Optimizer: AdamW (requires_grad=True인 파라미터만 포함)
- Batch Size: 8
- Epochs: 52
- Loss Function: CrossEntropyLoss (ignore_index=tokenizer.pad_token_id 적용), 실험_1_v_4에서는 R-Drop Loss (KL + CE) 도입
- LoRA 설정: rank=4, alpha=16

학습 전략

1. 초기 설정: LoRA(9~11)만 학습
2. 점진적 확장: LoRA(9~11) → LoRA(9~11) + Layer(9~11) → LoRA(6~11) + Layer(6~11) 순으로 확장 또는 축소

3. Learning Rate: $1e-5 \rightarrow 5e-6 \rightarrow 1e-3$ 순으로 실험하며, local minimum 탈출을 시도
4. Epoch마다 Validation Loss 측정, 각 버전별로 중간 가중치를 저장 및 로드하여 실험 지속

실험 파이프라인 버전

- v0: LoRA(9~11)
- v1: LoRA(9~11), Optimizer 수정 + ignore padding 추가
- v2: Layer(9~11) + LoRA(9~11)
- v3.1: Layer(6~11) + LoRA(9~11)
- v3.2: Layer(9~11) + LoRA(9~11), lr= $1e-3$
- v3.3: Layer(11), lr= $1e-3$
- v3.4: Layer(9~11) + LoRA(9~11), lr= $1e-5$ 복구
- v4: R-Drop Loss (KL + CE) 도입

학습 로그 요약

- v0~v2: 점진적 성능 향상 → 표현력 확장 효과 확인
- v3.1~v3.4: Validation Loss가 Train Loss와 Decoupling되며 과적합 발생
- v4: Loss function을 변경하였으나, 과적합 완화에 실패함 → 데이터셋 부족이 원인으로 판단됨

학습 로그

__version 0__

```
{date: 06-01, version: 0, epoch: 1, loss: 5.291}
{date: 06-01, version: 0, epoch: 5, loss: 5.281}
{date: 06-01, version: 0, epoch: 10, loss: 5.266}
{date: 06-01, version: 0, epoch: 15, loss: 5.221}
{date: 06-01, version: 0, epoch: 20, loss: 5.145}
```

- LoRA (9~12)만 학습 가능하게 설정.

- 20 epoch 동안 loss가 0.146만 줄어드는 등 성능 개선이 미미함
- 3모델의 표현력이 부족하다고 판단.

__version 1__

{date: 06-07, version: 1, epoch: 1, train_loss: 5.205, validation_loss: 5.198}

{date: 06-07, version: 1, epoch: 5, train_loss: 5.193, validation_loss: 5.194}

{date: 06-07, version: 1, epoch: 10, train_loss: 5.189, validation_loss: 5.187}

- 과적합을 판단하기 위해 validation loss를 추가.
- optimizer 수정 (requires_grad=True 파라미터만 적용).
- 패딩 토큰을 포함하지 않도록 loss 계산 방식 수정.
- 학습 없이도 손실값이 크게 감소 (5.291 → 5.205): 이전 설정 오류였을 가능성.
- 여전히 성능 향상은 제한적.

__version 2__

{date: 06-07, version: 2, epoch: 0, train_loss: 5.175, validation_loss: 5.136}

{date: 06-07, version: 2, epoch: 5, train_loss: 4.947, validation_loss: 4.960}

{date: 06-07, version: 2, epoch: 10, train_loss: 4.819, validation_loss: 4.860}

{date: 06-07, version: 2, epoch: 15, train_loss: 4.730, validation_loss: 4.783}

{date: 06-07, version: 2, epoch: 20, train_loss: 4.646, validation_loss: 4.736}

{date: 06-07, version: 2, epoch: 25, train_loss: 4.614, validation_loss: 4.706}

- layer(9~11)와 LoRA(9~11)를 모두 학습하도록 설정하여 모델 복잡도 증가.
- 성능이 크게 향상됨 (v1 대비 10 epoch 시 train_loss: 5.189 → 4.819).
- 그러나 epoch 20 이후 loss 감소 폭이 줄어들음 -> 모델 복잡도가 여전히 부족하다고 판단.

__version 3__

{date: 06-07, version: 3.1, epoch: 25, train_loss: 4.614, validation_loss: 4.706}

{date: 06-07, version: 3.1, epoch: 26, train_loss: 4.686, validation_loss: 4.733}

{date: 06-07, version: 3.1, epoch: 30, train_loss: 4.490, validation_loss: 4.623}

{date: 06-07, version: 3.1, epoch: 35, train_loss: 4.365, validation_loss: 4.572}

{date: 06-07, version: 3.1, epoch: 40, train_loss: 4.292, validation_loss: 4.543}

{date: 06-07, version: 3.1, epoch: 45, train_loss: 4.200, validation_loss: 4.528}

- v_2에서 25 epoch 학습하고 난 뒤의 가중치를 이어서 학습.
- v_3.1은 layer(6~11) + LoRA(9~11)로 모델의 complexity를 더 높였다.
- 다시 빠르게 loss가 떨어지는 것을 확인.
- epoch 35 이후 train과 validation loss의 디커플링 → 과적합 발생 판단

{date: 06-07, version: 3.2, epoch: 36, train_loss: 4.664, validation_loss: 4.546}

{date: 06-07, version: 3.2, epoch: 37, train_loss: 4.084, validation_loss: 4.569}

{date: 06-07, version: 3.2, epoch: 38, train_loss: 3.619, validation_loss: 4.669}

- layer(9~11) + LoRA(9~11).
- 과적합이 진행되기 전인 epoch 35 학습하고 난 뒤의 가중치를 이어서 학습한다.
- 과적합을 극복하기 위해 가중치를 $1e-5 \rightarrow 1e-3$ 으로 설정했다. (실험적 접근).
- 이는 local minimum을 피하기 위함이었다.
- 그러나 과적합이 계속 진행되었다.

{date: 06-07, version: 3.3, epoch: 36, train_loss: 4.593, validation_loss: 4.513}

{date: 06-07, version: 3.3, epoch: 37, train_loss: 4.108, validation_loss: 4.530}

{date: 06-07, version: 3.3, epoch: 38, train_loss: 3.724, validation_loss: 4.614}

{date: 06-07, version: 3.3, epoch: 39, train_loss: 3.316, validation_loss: 4.792}

- 이번엔 layer를 11만 학습할 수 있게 설정하였다.
- 마찬가지로 과적합이 진행되기 전인 epoch 35 학습하고 난 뒤의 가중치를 이어서 학습한다.
- 과적합을 극복하기 위해 가중치를 $1e-5 \rightarrow 1e-3$ 으로 설정했다. (실험)

- 지금까지 기록 중 가장 훌륭한 성적이 나왔다. Validation_loss가 4.513 달성했다.
- 그러나 그 이후로 과적합은 더욱 심하게 진행되었다.

{date: 06-07, version: 3.4, epoch: 37, train_loss: 3.988, validation_loss: 4.520}

{date: 06-07, version: 3.4, epoch: 38, train_loss: 4.155, validation_loss: 4.520}

{date: 06-07, version: 3.4, epoch: 39, train_loss: 4.122, validation_loss: 4.518}

{date: 06-07, version: 3.4, epoch: 40, train_loss: 4.101, validation_loss: 4.513}

{date: 06-07, version: 3.4, epoch: 41, train_loss: 4.099, validation_loss: 4.512}

- layer(9~11) + LoRA(9~11), lr = 1e-5
- v_3.3 epoch 36의 가중치 사용.
- 학습률을 다시 1e-3 -> 1e-5로 설정하였다.
- 학습률을 크게 키워서 분포가 크게 변했을 수 있었음에도, 다시 잘 학습하는 것을 확인하였다.
- 그럼에도 과적합 문제를 피해갈 수 없었다.

__version 4__

{date: 06-07, version: 4, epoch: 37, train_loss: 4.284, validation_loss: 4.497}

{date: 06-07, version: 4, epoch: 42, train_loss: 4.224, validation_loss: 4.489}

{date: 06-07, version: 4, epoch: 47, train_loss: 4.525, validation_loss: 4.581}

{date: 06-07, version: 4, epoch: 52, train_loss: 4.045, validation_loss: 4.577}

- layer(9~11) + LoRA(9~11), R-Drop Loss (KL + CE) 도입.
- v_3.3의 epoch 36의 가중치를 이어서 학습했다.
- 똑같이 과적합 현상이 나타났다.
- 결국 과적합 문제를 해결하지 못했다.
- 데이터셋의 부족이 근본 원인으로 판단, distillation 기법을 도입.

결과 분석 및 결론

- 학습 안정성: 점진적 튜닝 전략은 과적합 방지를 위한 유효한 방식으로 작용하였으나, 특정 시점을 넘어서면 성능이 정체되거나 악화됨
- 표현력 한계 해결: LoRA만 적용한 초기 세팅보다 Transformer 레이어를 함께 학습한 설정에서 명확한 성능 향상을 확인
- 과적합 문제: 학습 후반부로 갈수록 Validation Loss가 정체 또는 증가하며 과적합 발생. R-Drop Loss 도입, 레이어 축소, lr 조절 등 다각도로 접근했으나 완전한 해결에는 실패함
- 결론: 모델 구조와 학습 전략은 합리적이거나, 데이터셋의 양과 다양성이 부족하여 성능 상한선이 존재함.
- 다음 단계: distillation을 통해 데이터셋을 증강하는 것이 핵심 과제로 도출됨

2.3.3 실험 2 - Distillation 적용 실험

데이터 부족으로 인한 과적합 문제를 해결하기 위해, LLM(GPT-4.1)을 이용한 Distillation 데이터셋을 도입하여 학습 성능을 향상시키는 것을 목표로 한다.

실험 설계:

1. Distillation 데이터셋 생성
 - 1.4에서 기술한 방식으로 데이터셋을 생성한다.
2. Distillation 데이터셋으로 파인 튜닝
3. Ground-truth 데이터셋과 병행하며 파인 튜닝
4. 마지막 학습에선 Ground-truth only 파인 튜닝

가정

- Distillation 데이터를 먼저 사용하고, 이후 ground-truth 데이터로 점진적으로 전환하는 방식은 노이즈를 줄이고 모델의 안정화와 일반화 성능 향상에 기여할 것으로 기대된다.
- LLM 기반 생성 데이터는 대량 확보가 가능하므로, 초기 과적합을 방지하고 표현 학습의 기반을 넓힐 수 있다.
- 마지막 단계에서 ground-truth만을 사용하는 것은 모델의 최종 품질 보정 역할을 한다.

파이프라인

sonnet_generation.py의 main() 함수는 미리 정의된 파이프라인을 순회하면서 각 단계별로 start_epoch, end_epoch, freeze_lora_layers, unfreeze_blocks, dataset을 설정하고, 이전 단계에서 학습된 가중치를 그대로 이어받아 다음 단계로 넘어간다.

초기 단계에는 비교적 소수의 LoRA 레이어만 unfreeze한 채로 distilled 데이터셋을 사용해 학습을 진행하고, 이후 단계에서는 점차 더 많은 LoRA 레이어를 unfreeze한 상태로 원본 sonnet 데이터셋과 병행하여 모델을 fine-tuning한다. 이러한 구조 덕분에 간단한 표현 학습에서 시작해 점진적으로 모델의 전체 표현력을 확장해 나갈 수 있다.

```
if __name__ == "__main__":
    args = get_args()
    seed_everything(args.seed) # 재현성을 위한 random seed 고정.
    pipeline = [
        {
            "version": 0,
            "start_epoch": 0,
            "end_epoch": 30,
            "freeze_lora_layers": [0,1,2,3,4,5,6,7,8,9],
            "unfreeze_blocks": [10,11],
            "dataset": "distilled"
        },
        {
            "version": 1,
            "start_epoch": 30,
            "end_epoch": 60,
            "freeze_lora_layers": [0,1,2,3,4,5],
            "unfreeze_blocks": [8,9,10,11],
            "dataset": "distilled"
        },
        {
            "version": 2,
            "start_epoch": 60,
            "end_epoch": 140,
            "freeze_lora_layers": [],
            "unfreeze_blocks": [8,9,10,11],
            "dataset": "distilled"
        },
        {
            "version": 3,
            "start_epoch": 200,
            "end_epoch": 220,
            "freeze_lora_layers": [],
            "unfreeze_blocks": [6,7,8,9,10,11],
            "dataset": "distilled"
        },
        {
            "version": 4,
            "start_epoch": 220,
            "end_epoch": 280,
            "freeze_lora_layers": [0,1,2,3,4,5,6,7,8,9],
            "unfreeze_blocks": [10,11],
            "dataset": "sonnet"
        },
        {
            "version": 5,
            "start_epoch": 280,
            "end_epoch": 310,
            "freeze_lora_layers": [0,1,2,3,4,5],
            "unfreeze_blocks": [8,9,10,11],
            "dataset": "sonnet"
        },
        {
            "version": 6,
            "start_epoch": 310,
            "end_epoch": 390,
            "freeze_lora_layers": [],
            "unfreeze_blocks": [6,7,8,9,10,11],
            "dataset": "distilled"
        },
        {
            "version": 7,
            "start_epoch": 390,
            "end_epoch": 430,
            "freeze_lora_layers": [],
            "unfreeze_blocks": [6,7,8,9,10,11],
            "dataset": "sonnet"
        },
        {
            "version": 8,
            "start_epoch": 430,
            "end_epoch": 480,
            "freeze_lora_layers": [],
            "unfreeze_blocks": [0,1,2,3,4,5,6,7,8,9,10,11],
            "dataset": "distilled"
        },
        {
            "version": 9,
            "start_epoch": 480,
            "end_epoch": 530,
            "freeze_lora_layers": [],
            "unfreeze_blocks": [0,1,2,3,4,5,6,7,8,9,10,11],
            "dataset": "sonnet"
        }
    ],
```

위와 같이 파이프라인을 설정하였다.

Pipeline list를 for conf in pipeline: 으로 순회하면서, 각 요소인 딕셔너리(conf)를 train_conf(args, **conf) 형태로 키워드 인자(**conf)로 풀어 전달한다.

```
for conf in pipeline:
    ckpt_path = train_conf(args, **conf)
    print(f"[✓] {conf['version']} 완료: {ckpt_path}")
```

train_con함수는 다음과 같이 정의되어 있다.

```
def train_conf(
    args,
    version,
    start_epoch,
    end_epoch,
    freeze_lora_layers=None,
    unfreeze_blocks=None,
    dataset=None
):
    args.freeze_lora_layers = freeze_lora_layers or []
    args.unfreeze_blocks = unfreeze_blocks or []
    if dataset == "distilled":
        args.sonnet_path = "data/Full_gpt4_distillation_sonnet_outputs.txt"
        print('distilled')
    elif dataset == "sonnet":
        args.sonnet_path = "data/sonnets.txt"
        print('sonnet')

    else:
        raise Exception(f'{dataset} is not supported.')
    args.filepath = f'{start_epoch}-sonnet.pt'
    args.epochs = end_epoch
    train(args)
    generate_submission_sonnets(args)
```

학습 로그

__version 0__

{epoch: 0, train_loss: 5.812, val_loss: 6.107}

{epoch: 6, train_loss: 4.548, val_loss: 5.605}

{epoch: 11, train_loss: 4.204, val_loss: 5.754}

{epoch: 16, train_loss: 4.040, val_loss: 5.893}

{epoch: 21, train_loss: 3.929, val_loss: 5.978}

{epoch: 26, train_loss: 3.844, val_loss: 6.033}

Test_score: 57.71

- distillation dataset으로 순조롭게 학습 중이다.

- validation loss가 늘어나고 있지만, distillation 데이터의 품질을 신뢰하기 때문에 학습을 계속 진행한다.

- distillation dataset으로 학습한 뒤 마지막엔 gold 데이터셋으로 전이 학습할 예정이다.

__version 1__

{epoch: 31, train_loss: 3.765, val_loss: 6.043}

{epoch: 36, train_loss: 3.611, val_loss: 6.020}

{epoch: 41, train_loss: 3.526, val_loss: 6.033}

{epoch: 46, train_loss: 3.454, val_loss: 6.075}

{epoch: 51, train_loss: 3.391, val_loss: 6.113}

{epoch: 56, train_loss: 3.337, val_loss: 6.131}

Test_score: 58.28

- 학습할 수 있는 Transformer block과 RoLA Layer를 더 unfreeze하였다.

- 모델의 complexity가 늘어났고, 느려지던 학습 속도가 다시 빨라졌다.

__version 2__

{epoch: 61, train_loss: 3.283, val_loss: 6.162}
{epoch: 66, train_loss: 3.244, val_loss: 6.193}
{epoch: 71, train_loss: 3.202, val_loss: 6.239}
{epoch: 76, train_loss: 3.165, val_loss: 6.273}
{epoch: 81, train_loss: 3.129, val_loss: 6.295}
{epoch: 86, train_loss: 3.093, val_loss: 6.326}
{epoch: 91, train_loss: 3.069, val_loss: 6.342}
{epoch: 96, train_loss: 3.036, val_loss: 6.364}
{epoch: 101, train_loss: 3.008, val_loss: 6.405}
{epoch: 106, train_loss: 2.986, val_loss: 6.435}
{epoch: 111, train_loss: 2.959, val_loss: 6.456}
{epoch: 116, train_loss: 2.934, val_loss: 6.467}
{epoch: 121, train_loss: 2.914, val_loss: 6.499}
{epoch: 126, train_loss: 2.896, val_loss: 6.502}
{epoch: 131, train_loss: 2.877, val_loss: 6.508}
{epoch: 136, train_loss: 2.848, val_loss: 6.571}

Test_score: 58.78

- RoLA는 모든 레이어를 학습 중이고, Transformer Block은 4개 블록이 학습 중이다.
- train_loss는 0.5나 줄었으나, Test_socre는 0.5밖에 늘지 않았다.

__version 3__

{epoch: 141, train_loss: 2.811, val_loss: 6.554}
{epoch: 146, train_loss: 2.738, val_loss: 6.542}
{epoch: 151, train_loss: 2.695, val_loss: 6.576}
{epoch: 156, train_loss: 2.662, val_loss: 6.601}

{epoch: 161, train_loss: 2.628, val_loss: 6.612}

{epoch: 166, train_loss: 2.594, val_loss: 6.635}

{epoch: 171, train_loss: 2.569, val_loss: 6.668}

{epoch: 176, train_loss: 2.544, val_loss: 6.679}

{epoch: 181, train_loss: 2.517, val_loss: 6.681}

{epoch: 186, train_loss: 2.487, val_loss: 6.762}

{epoch: 191, train_loss: 2.469, val_loss: 6.742}

{epoch: 196, train_loss: 2.445, val_loss: 6.755}

{epoch: 201, train_loss: 2.416, val_loss: 6.832}

{epoch: 206, train_loss: 2.389, val_loss: 6.851}

{epoch: 211, train_loss: 2.368, val_loss: 6.909}

{epoch: 216, train_loss: 2.349, val_loss: 6.915}

Test_score: 59.06

- 임의로 정한 목표인 'train_loss = 2.2'에 거의 근접했다.
- 다음 학습은 sonnet.txt로 학습할 것이다.

__version 4__

{epoch: 221, train_loss: 7.015, val_loss: 6.553}

{epoch: 226, train_loss: 5.649, val_loss: 5.438}

{epoch: 231, train_loss: 5.316, val_loss: 5.093}

{epoch: 236, train_loss: 5.188, val_loss: 4.937}

{epoch: 241, train_loss: 5.111, val_loss: 4.848}

{epoch: 246, train_loss: 5.052, val_loss: 4.791}

{epoch: 251, train_loss: 5.005, val_loss: 4.753}

{epoch: 256, train_loss: 4.959, val_loss: 4.725}

{epoch: 261, train_loss: 4.938, val_loss: 4.704}

{epoch: 266, train_loss: 4.912, val_loss: 4.687}

{epoch: 271, train_loss: 4.889, val_loss: 4.675}

{epoch: 276, train_loss: 4.859, val_loss: 4.664}

Test_score: 55.21

- sonnet.txt 데이터로 학습하니 Test_score가 크게 감소하였다.
- train_loss가 처음엔 pre_trained 모델보다 높았으나, 빠르게 loss가 줄어드는 걸 관찰할 수 있다.
- train_loss보다 val_loss가 더 작은 것을 관찰했다.

__version 5__

{epoch: 281, train_loss: 4.848, val_loss: 4.645}

{epoch: 286, train_loss: 4.746, val_loss: 4.606}

{epoch: 291, train_loss: 4.709, val_loss: 4.584}

{epoch: 296, train_loss: 4.682, val_loss: 4.572}

{epoch: 301, train_loss: 4.632, val_loss: 4.564}

{epoch: 306, train_loss: 4.590, val_loss: 4.558}

Test_score: 55.01

- sonnet.txt로 학습한 결과다.
- Test_score가 또 0.2점 감소했다.

__version 6__

{epoch: 311, train_loss: 2.797, val_loss: 5.299}

{epoch: 316, train_loss: 2.346, val_loss: 6.177}

{epoch: 321, train_loss: 2.313, val_loss: 6.343}

{epoch: 326, train_loss: 2.294, val_loss: 6.429}

{epoch: 331, train_loss: 2.271, val_loss: 6.514}

{epoch: 336, train_loss: 2.252, val_loss: 6.578}

{epoch: 341, train_loss: 2.235, val_loss: 6.629}

{epoch: 345, train_loss: 2.218, val_loss: 6.662}

{epoch: 351, train_loss: 2.199, val_loss: 6.714}

{epoch: 356, train_loss: 2.180, val_loss: 6.770}

{epoch: 361, train_loss: 2.163, val_loss: 6.808}

{epoch: 366, train_loss: 2.143, val_loss: 6.870}

{epoch: 371, train_loss: 2.131, val_loss: 6.890}

{epoch: 376, train_loss: 2.116, val_loss: 6.928}

{epoch: 381, train_loss: 2.094, val_loss: 6.944}

{epoch: 386, train_loss: 2.079, val_loss: 7.007}

Test_score: 59.87

- sonnet.txt 데이터로 90 epoch을 학습한 뒤 다시 distillation 데이터로 학습했는데 train_loss가 잘 보존되어 있는 걸 관찰했다.

- Test_score 최고점을 달성했다.

__version 7__

{epoch: 391, train_loss: 5.976, val_loss: 5.272}

{epoch: 396, train_loss: 4.726, val_loss: 4.627}

{epoch: 401, train_loss: 4.624, val_loss: 4.576}

{epoch: 406, train_loss: 4.581, val_loss: 4.553}

{epoch: 411, train_loss: 4.542, val_loss: 4.538}

{epoch: 416, train_loss: 4.505, val_loss: 4.525}

{epoch: 421, train_loss: 4.467, val_loss: 4.519}

{epoch: 426, train_loss: 4.422, val_loss: 4.517}

Test_score: 55.70

- distillation 데이터로 80epoch을 학습한 뒤 다시 sonnet.txt 데이터로 학습했다.

- version 6와 달리 train_loss가 많이 커진 것을 관찰했다.

__version 8__

{epoch: 431, train_loss: 2.270, val_loss: 5.445}

{epoch: 436, train_loss: 1.882, val_loss: 6.202}

{epoch: 441, train_loss: 1.826, val_loss: 6.448}

{epoch: 446, train_loss: 1.782, val_loss: 6.610}

{epoch: 451, train_loss: 1.740, val_loss: 6.721}

{epoch: 456, train_loss: 1.699, val_loss: 6.788}

{epoch: 461, train_loss: 1.661, val_loss: 6.913}

{epoch: 466, train_loss: 1.621, val_loss: 7.031}

{epoch: 471, train_loss: 1.584, val_loss: 7.164}

{epoch: 476, train_loss: 1.543, val_loss: 7.256}

Test_score: 59.61

- 모든 Transformer block과 RoLA Layer를 unfreeze했다.
- distillation data의 train_loss가 1.543까지 빠르게 감소했다.

__version 9__

{epoch: 481, train_loss: 5.340, val_loss: 4.990}

{epoch: 486, train_loss: 4.240, val_loss: 4.482}

{epoch: 491, train_loss: 4.123, val_loss: 4.461}

{epoch: 496, train_loss: 4.026, val_loss: 4.452}

{epoch: 501, train_loss: 3.956, val_loss: 4.453}

Test_score: 56.17

- 모든 Transformer block과 RoLA Layer를 unfreeze했다.
- train_loss와 val_loss 둘 다 빠르게 작아지는 것을 관찰할 수 있다.

결과 분석 및 결론

본 실험에서는 distillation 데이터셋을 활용한 사전 학습과 gold 데이터셋(sonnet.txt)을 활용한 전이 학습을 통해 GPT-2 기반 Sonnet 생성 모델을 점진적으로 파인튜닝하였다. 총 10개의 학습 버전(version 0~9)을 설계하여, RoLA Layer와 Transformer Block의 학습 범위를 단계적으로 확장하는 파이프라인 구조로 실험을 진행하였다.

초기에는 distillation 데이터셋으로 안정적인 수렴을 유도하였으며, RoLA와 Transformer Layer를 점진적으로 unfreeze함으로써 모델의 복잡도를 확장하였다. 그 결과 train loss는 지속적으로 감소했으나, validation loss는 다소 정체된 경향을 보였다. 이후 gold 데이터셋(sonnet.txt)으로 전이 학습을 수행하면서 초기에는 train loss가 급증했으나 빠르게 감소하며 전이 학습의 효과를 확인할 수 있었다.

특히 version 6와 version 8에서는 distillation 데이터로 학습을 재개한 결과, train loss는 낮은 상태를 유지하며, Test Score에서 각각 59.87, 59.61로 가장 높은 성능을 기록하였다. 반면, gold 데이터셋만으로 학습을 지속하거나 전환 순서를 바꾸었을 때 test score가 감소하거나 불안정한 양상을 보였다.

결과적으로, 본 실험은 데이터셋의 순차적 전환과 모델 복잡도의 점진적 확장이 성능 향상에 효과적이라는 것을 보여주었으며, 특히 distillation → gold → distillation의 흐름이 가장 안정적이고 높은 성능을 달성하는 전략임을 실험적으로 검증하였다.